

Java Swing and AWT Study Notes

Based on the Student Gradebook project

These notes explain the GUI APIs used in `GradeSystemApp.java`, how the screen is arranged, and how the gradebook reacts to user input. The application uses Swing widgets, AWT layout and styling classes, and a small object-oriented student model.

1. Swing and AWT: how they fit together

AWT (Abstract Window Toolkit) is Java's original GUI foundation. It provides window-system support, graphics primitives, colors, fonts, dimensions, and layout managers. Swing is a richer GUI toolkit built on top of AWT. Swing components usually begin with J, such as `JFrame`, `JPanel`, `JButton`, and `JTable`; AWT classes such as `BorderLayout`, `Color`, and `Font` are still commonly used with them.

In this project, AWT is mainly used for layout and appearance, while Swing supplies the visible interface components. The student model itself does not depend on either GUI toolkit.

2. Main Swing components in the project

Class	Role in this gradebook
<code>JFrame</code>	The top-level application window. <code>GradeSystemApp</code> extends it.
<code>JPanel</code>	A container used to group the header, summary cards, form, and report.
<code>JLabel</code>	Displays headings, field captions, status, student count, and summary values.
<code>TextField</code>	Accepts the ID, name, and marks typed by the user.
<code>JButton</code>	Runs the Add student and Reset all actions.
<code>JTable</code>	Displays one row per student, with ID, name, marks, and grade.
<code>DefaultTableModel</code>	Stores the table's rows and column headings. The project overrides <code>isCellEditable</code> to make report cells read-only.
<code>JScrollPane</code>	Adds a scrollable viewport around the report table.
<code>UIManager</code>	Selects the operating system's Swing look and feel when available.
<code>SwingUtilities</code>	Starts GUI creation on Swing's event-dispatch thread.
<code>DefaultTableCellRenderer</code>	Customizes table-cell alignment, font, background, and grade color.

3. AWT layout managers used

A layout manager positions components when a window is resized. It is generally preferable to fixed pixel coordinates.

BorderLayout

Divides a container into NORTH, SOUTH, EAST, WEST, and CENTER regions. The app uses it for the main window and for cards that place a title, content, or action area in named regions.

A region usually holds one component; use a nested JPanel when a region needs several items.

GridBagLayout and GridBagConstraints

Places components in a flexible grid. Each component's GridBagConstraints describes its gridx/gridy position, weightx expansion, fill behavior, and insets spacing. The app uses it for the four summary cards and for the three input fields. weightx = 1 and fill = HORIZONTAL let the fields share available width.

BoxLayout

Arranges components in one direction. The page uses Y_AXIS to stack the header, statistics, form, and report. Small rigid areas provide vertical gaps.

Insets and dimensions

Insets sets the space around a component within a grid cell. Dimension describes preferred or minimum width and height. BorderFactory creates line, empty, compound, and matte borders; combining a line border with an empty border gives a clean outline plus inner padding.

4. Events: making buttons do work

A GUI is event-driven: it waits for user actions and then calls a handler. The Add button registers an action listener with a lambda, conceptually like this:

```
addButton.addActionListener(event -> addStudent());
```

The Reset button similarly calls resetAll(). Keep event handlers short and move meaningful work into named methods. In this project, addStudent() reads and validates the fields, creates a Student, adds a row, and updates the summary. resetAll() clears both the model data and visible values.

5. Table model and custom rendering

DefaultTableModel separates the table's data from its display. The gradebook creates four columns, then adds each student as a row. Overriding isCellEditable prevents accidental editing of calculated report values.

A JTable renderer controls how an individual cell is painted. ReportCellRenderer centers ID, marks, and grade values, uses bold text for names and grades, and colors passing grades differently from an F. Rendering changes appearance only; it should not modify the underlying data.

6. The gradebook's data model and rules

Person is an abstract parent class containing a name and ID. Student extends Person and adds marks. This demonstrates encapsulation (private fields with validated setters), inheritance (Student extends Person), and abstraction (the base class represents shared person details).

The grade thresholds are implemented in `Student.calculateGrade()`:

Marks	Grade
90 to 100	A
80 to less than 90	B
70 to less than 80	C
60 to less than 70	D
0 to less than 60	F

`setMarks` rejects values outside 0-100. The form additionally rejects invalid numeric input and duplicate student IDs. The report is limited to five students, matching the original console program.

7. Summary calculations

After each successful addition, the application updates the class size, average, number passing, and top grade. For n students with marks m_i , the average is:

Average = sum of all marks divided by the number of students.

The passing count includes grades A, B, C, and D; F is not passing. Since A is the highest grade, the top grade is the best (earliest in A-to-F order) grade present in the class.

8. Swing's event-dispatch thread

Swing components are generally not thread-safe. Create and show the interface on the event-dispatch thread (EDT):

```
SwingUtilities.invokeLater(() -> {  
    new GradeSystemApp().setVisible(true);  
});
```

Short event handlers are safe to run there. Long-running work should be moved off the EDT (for example, with `SwingWorker`) so the window does not freeze.

9. Colors, fonts, and look and feel

Color represents component and text colors. Font sets the family, style (such as BOLD or PLAIN), and point size. Components such as `JLabel` and `JButton` provide methods like `setForeground`, `setBackground`, and `setFont`.

`UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName())` asks Swing to use the operating system's native style. The app catches a failure and continues with Swing's default appearance. A look and feel changes the visual theme, not the application's grade rules.

10. How the interface works, end to end

- The main method schedules window setup on the EDT.
- The constructor configures the frame and builds panels for the header, statistics, entry form, and report.
- The user types an ID, name, and mark, then selects Add student.

- The handler parses input; Student validates the data and calculates a grade.
- The student is added to the list and the table model; labels are recalculated.
- Reset all clears the current report and enables another set of up to five entries.

11. Practical study tips

- * Change one layout manager at a time and observe how resizing behaves.
- * Try boundary marks: 59, 60, 69, 70, 79, 80, 89, 90, and 100.
- * Test invalid marks, blank names, non-numeric input, and duplicate IDs.
- * Add five students and verify that Add is disabled; then reset and confirm it is enabled again.
- * Keep business rules in Student and interface behavior in GradeSystemApp so the model can be tested without opening a window.
- * Build Swing screens with nested panels and layout managers instead of absolute positioning.